CS260C: Deep Learning **Spring 2026**

Homework 1: Due May 10, 11:59 PM (No Late Submission)

For this homework, we provide a skeleton code as a Google Colab notebook: `https://colab.research.google.com/drive/1mnOr5KxJhXLT_iL0I0awAgwyBjTZidmv?usp=sharing`. You need to work on the notebook, and implement the rest of the code. You may either work with Google Colab online (you need to work on your own copy of the notebook), or download the notebook to your own device and work with a local Jupyter Notebook environment.

Part One: Multi-class Image Classification (30pt)

In this part, you will practice building a multi-class image classification pipeline to classify images (from MNIST and CIFAR-10 datasets) using fully-connected neural networks (MLP). The goals of are as follows:

- Understand the pipeline of building multi-class image classifiers.
- Implement and apply a fully-connected neural network classifier.

Requirements

Implement fully-connected neural network (MLP) image classifiers on MNIST and CIFAR-10 datasets.

1. Defining one fully-connected neural network model for each dataset;
   (a) MNIST: a **3-layer** model (3 `torch.nn.Linear` layers, where 2 of them are hidden layers) with **ReLU** activation. Each hidden layer should have **512** neurons.
   (b) CIFAR-10: a **3-layer** model with **ReLU** activation. Each hidden layer should have **1024** neurons

   The size of the last linear layer should be equal to the number of classes.
2. Running training and optimization;
   (a) Use SGD optimizer with momentum. Compare and choose a good **learning rate**, from: $\{10^{-3}, 10^{-2}, 0.1, 0.2, 0.5, 1.0, 5.0\}$. Compare and choose a good **momentum factor**, from: $\{0, 0.1, 0.2, 0.5, 0.8, 0.9, 1.0\}$.
   (b) When using PyTorch's SGD optimizer, please do not set other hyperparamters except for learning rate and momentum (use default values for other hyperparameters).
   (c) Train for at least 10 epochs.
3. Evaluating and reporting the test accuracy.
   (a) Compute and print the accuracy **on the test data after each epoch**.
   (b) You are expected to achieve at least **98%** test accuracy on MNIST and **55%** test accuracy on CIFAR-10.

Part two - Convolutional Neural Networks

In this part, practice implementing convolutional layers, and train convolutional neural networks on MNIST and CIFAR-10 for image classification. The goals of are as follows:

- Understand the computation of convolution layers.
- Practice implementing convolutional layers efficiently.
- Apply convolutional neural networks for image classification.

2.1 Implementing Convolution (30pt)

In this part, you are asked to implement convolution in different ways in the coding notebook. We will assume the batch size is just 1. **You are not allowed to use high-level PyTorch functions/classes directly for convolution**, including: `F.conv2d`, `F.conv1d`, `nn.conv2d`, `nn.Conv1d`, `F.unfold`, `F.pad`, and potentially other similar functions. There are two functions to be implemented:

- `conv_loop` **(12pt)**: Use the definition of convolution layer, and use nested loops to implement the convolution computation in a straightforward way. 6 levels of nested loops are expected (input channels, output channels, output height, output width, kernel height, and kernel width).
- `conv` **(18pt)**: It is inefficient to use too many nested loops to access tensors in PyTorch or make too many calls to PyTorch functions/operators (in particular, multiplication here for implementing convolution). Thus, you are also asked to implement another version of convolution while trying to use as fewer levels of loops as possible. Implementation should be correct and pass test cases in the coding notebook.

  You will get 3 points for every nested loop you reduce from the `conv_loop` baseline, while effectively reducing the number of PyTorch calls, i.e., you cannot hack the scoring rule by simply replacing loops with something else that is equivalently still doing loops and does not actually reduce the number of PyTorch calls. If your correct implementation does not have any loop (and number of PyTorch calls is $O(1)$), you get all 18 points.

Hints A possible way:

- Construct a matrix/tensor, where each row consists of the **input indexes for each sliding window** (Some functions that might be helpful: `torch.arange`, `torch.cat`, `torch.permute`).
- Fetch the input values of each sliding window, using the indexes (For input tensor `x` and index matrix/tensor `index`, the corresponding values can be obtained via `x[index]`);
- Multiply the input values with the kernels, and accumulate/rearrange the results.

2.2 Training CNN on MNIST and CIFAR-10 (40pt)

In this part, you are asked to train image classifiers on MNIST and CIFAR-10 with CNNs.

1. Defining one CNN model for each dataset. For all the convolution layers, use kernel_size = 3, padding = 1. The hidden layers are as follows:

   (a) MNIST:
   - Layer 1: Convolution layer, with 32 output channels and stride = 1.
   - Layer 2: Convolution layer, with 64 output channels and stride = 2.
   - Layer 3: Fully-connected layer, with 512 output neurons.

   (b) CIFAR-10:
   - Layer 1: Convolution layer, with 32 output channels and stride = 1.
   - Layer 2: Convolution layer, with 64 output channels and stride = 2.
   - Layer 3: Convolution layer, with 128 output channels and stride = 2.
   - Layer 4: Convolution layer, with 256 output channels and stride = 2.
   - Layer 5: Fully-connected layer, with 512 output neurons.

   For convolutional layers, you may use `torch.nn.Conv2d` or the one implemented by yourself. There needs to be a **ReLU activation** after each hidden layer. In the end, there is also an additional layer with output size equal to the number of classes (no ReLU activation for this layer).

2. Running training and optimization;

   (a) Try two different optimizers:
   - **SGD** optimizer with momentum. Choose a good learning rate, from: $\{0.0001, 0.0005, 0.001, 0.005, 0.01, 0.05, 0.1, 0.5\}$. Choose a good momentum factor, from: $\{0, 0.9\}$.
   - **Adam** optimizer. Choose a good learning rate from $\{0.0001, 0.0005, 0.001, 0.005, 0.01, 0.05, 0.1, 0.5\}$.

   Choose the best hyperparameter setting for each optimizer respectively, and compare the two optimizers. You don't need to report the hyperparameter choice for each optimizer.

(b) Train for at least 20 epochs.

3. Evaluating and reporting the test accuracy.

   (a) Compute and print the accuracy **on the test data after each epoch**.

   (b) You are expected to achieve at least **99%** test accuracy on MNIST and **70%** test accuracy on CIFAR-10.

Hint It can be slow to train CNN using CPU. You may use GPU runtime on Google Colab (Runtime→Change runtime type). If you decide to use GPU, you need to move PyTorch models and tensors (input and label) to CUDA by calling `.cuda()`. See: `https://pytorch.org/docs/stable/notes/cuda.html`.

Submission

- Submit your notebook to **Gradescope** and name it exactly as `hw1.ipynb`. Failure to follow these instructions will result in the autograder failing, which will automatically result in 0 points. No regrading will be done for submissions with incorrect file names or formats. **Please make sure to keep the output of your final run in the notebook.**

Resources

- PyTorch documentation: `https://pytorch.org/docs/stable/index.html`.
- Google Colab: `https://research.google.com/colaboratory`.
- SGD: `https://pytorch.org/docs/stable/generated/torch.optim.SGD.html`.
- MNIST: `http://yann.lecun.com/exdb/mnist/`.
- CIFAR-10: `https://www.cs.toronto.edu/~kriz/cifar.html`.